Detección temprana de comportamientos anómalos en redes de datos mediante modelos predictivos y un mecanismo de control en línea

"Detección temprana de comportamientos anómalos en redes de datos mediante aprendizaje automatico y un mecanismo de control en tiempo real"

"Detección temprana de comportamientos anómalos en redes de datos mediante aprendizaje automatico y un mecanismo de control inline"

Rubén Mark Salazar Tocas[1*] **Uziel Elias Sauñe Fernandez**[1]

ORCID 0009-0002-2386-9473 · ORCID 0009-0005-0223-0090

1 Escuela Profesional de Ingeniería de Sistemas, Facultad de Ingeniería y Arquitectura, Universidad Peruana Unión, Lima, Perú

* Correo del autor de correspondencia: ruben.salazar@upeu.edu.pe

Resumen: Se presenta CyberFlow, un sistema de código abierto para detectar anomalías de red y aplicar respuestas graduadas. El sensor observa tráfico IPv4 mediante un espejo SPAN y extrae 28 variables de red, transporte y aplicación. Combina un Isolation Forest congelado con cuatro heurísticos; las decisiones se distribuyen mediante un feed firmado y las restricciones temporales se aplican con nftables en el host protegido. En una evaluación reutilizada de 65 421 ventanas normales y 84 etiquetadas como ataque, el IF detectó 54/84, con precisión de 1,79 %, FPR de 4,53 %, ROC-AUC de 0,9247 y precisión media de 0,1858. Estos resultados describen el IF fuera de línea, no el desempeño del híbrido completo en operación. Se declaran el desbalance de clases, la incertidumbre de las etiquetas proxy y la necesidad de una prueba independiente.

Palabras clave: detección de intrusiones, detección de anomalías, aprendizaje no supervisado, Isolation Forest, control en línea, nftables, seguridad de redes

---

1. Introducción

La literatura reciente sobre sistemas de detección de intrusions en redes (Network Intrusion Detection Systems, NIDS) evolucionaron desde mecanismos basados en firmas hacia enfoques capaces de identificar comportamientos que son diferentes al funcionamiento esperado de una red. Los sistemas basados en firmas comparan el tráfico observado con patrones conocidos, de esta manera se logra buenos resultados frente a amenazas documentadas, pero presentan limitaciones cuando aparecen nuevas variants o ataques que no han sido registrados. Frente a todo esto varios enfoques buscan modelar el comportamiento normal del tráfico para identificar desviaciones potencialmente maliciosas, siendo una alternativa relevante para escenarios donde la información sobre ataques conocidos resulta insuficiente. Todo esto impulsó la aplicación de técnicas de aprendizaje automático y profundo en NIDS por su capacidad para analizar características complejas del tráfico y adaptarse a grandes volúmenes de datos. {1,2}

En paralelo al desarrollo de técnicas supervisadas, la literatura ha explorado enfoques no supervisados para detectar anomalías cuando el tráfico no dispone de un etiquetado exhaustivo. Estos métodos analizan los patrones presentes en los datos e identifican observaciones que se apartan del comportamiento predominante sin requerir clases de ataque previamente definidas. Dentro de este grupo, Isolation Forest (IF) ha sido aplicado a la detección de anomalías en tráfico de red mediante el aislamiento de observaciones atípicas, constituyendo una alternativa para analizar comportamientos desconocidos o poco frecuentes. {3}

Una limitación de los sistemas de detección basados en firmas es que su capacidad para reconocer una amenaza depende de que exista previamente un patrón o firma con el cual comparar el tráfico observado. Aunque este enfoque resulta efectivo frente a ataques conocidos, puede no identificar amenazas nuevas o variantes cuyo comportamiento aún no se encuentra registrado. Esta dependencia deja una brecha ante comportamientos desconocidos, por

lo que los enfoques basados en anomalías resultan relevantes al analizar desviaciones del comportamiento de la red sin requerir una firma específica para cada ataque. {4}

Además, los modelos de detección basados en anomalías enfrentan desafíos al trasladarse desde entornos simulados hacia redes operativas debido a que el tráfico real presenta variaciones constantes, comportamientos legítimos poco frecuentes y condiciones diferentes a las utilizadas durante el entrenamiento. Estas diferencias pueden incrementar la aparición de falsas alarmas y reducer la confiabilidad del Sistema cuando se enfrenta a datos no observados. Debido a esto el problema no se limita únicamente en mejorar la capacidad de clasificación, sino de desarrollar mecanismos que permitan detector desviaciones relevantes manteniendo un equilibrio entre sensibilidad, estabilidad operative y capacidad de respuesta. {5}

Frente a las limitaciones metodológicas expuestas, resulta necesario evaluar los IDS no solo por su capacidad para distinguir tráfico normal y anómalo, sino también por su funcionamiento al procesar tráfico de manera continua y la capacidad de generar una alerta o decision. Detectar amenazas desconocidas en tiempo real continua siendo un desafío debido a la evolución constante del tráfico y a la necesidad de mantener bajas tasas de falsas alarmas lo que motiva a generar propuestas referente al análisis continuo de datos de red {6}. También cuando la detección se asocia con mecanismos automáticos de respuesta, la decision debe ser controlada, ya que una decision incorrecta puede interrumpir procesos o comunicaciones legítimas. Investigaciones recientes sobre respuesta adaptative destacan precisamente la necesidad de integrar detección y mitigación depende del nivel de impacto operativo de las acciones ejecutadas{7}. Dicho esto resulta oportuno estudiar una arquitectura que capture, análisis de anomalías, decisiones y control del tráfico.

Este estudio tiene como objetivo diseñar, implementar y evaluar un sistema de detección temprana de comportamientos anómalos en redes de datos que integre un modelo Isolartion Forest (IF) para identificar desviaciones en el comportamiento del tráfico, junto con un mecanismo de decisión y control inline. La propuesta busca determinar si esta integración permite detectar comportamientos anómalos sin depender exclusivamente de firmas predefinidas y convertir la detección en una respuesta operativa sobre el tráfico, evaluando de manera conjunta el desempeño de detección, el lead time, la latencia de decisión y el impacto de las acciones de control sobre el tráfico legítimo en un entorno experimental.

En relación con esta brecha, los trabajos recientes muestran distintas aproximaciones para acercar la detección de intrusiones a escenarios operativos. Algunas de estas propuestas se centran en mejorar la selección de características y el desempeño del clasificador, pero mantienen su evaluación en esquemas de entrenamiento y prueba offline, dejando la detección en tiempo real como trabajo futuro {8}. Otras investigaciones vincularon la salida de un IDS con mecanismos defensivos activado por eventos, como la migración de máquinas virtuales, evidenciando que la respuesta automática también debe considerer su efecto sobre la disponibilidad y el desempeño del sistema {9}. También se desarrollaron arquitecturas que procesan alertas de Suricata provenientes de redes en producción y generan recomendaciones de respuesta para reducir el tiempo requerido para atender incidentes, auque la acción de ejecutar permanece como una etapa posterior {10}. Recientemente, se demostró la viabilidad de integrar extracción de características, inferencia mediante aprendizaje automático y bloqueo en tiempo real dentro de un gateway utilizando eBPF/XDP e iptables para poder aislar tráfico identificado como malicioso. {11}

Las contribuciones de este estudio son: (i) una arquitectura experimental reproducible que integra captura de telemetría, extracción causal de características multicapa y detección de anomalías mediante un modelo Isolation Forest orientado a identificar desviaciones en el comportamiento del tráfico; (ii) un mecanismo de decision y control que conecta la inferencia del modelo con acciones inline sobre el tráfico, reduciendo la separación entre detección y respuesta; y (iii) una evaluación que combina métricas predictivas con indicadores operativos como leadtime, latencia de decision y tasa de falsos positivos, permitiendo analizar no solo el

rendimiento del modelo en condiciones controladas, sino también su comportamiento al aproximarse a condiciones de operación.

El resto del presente artículo se organiza de la siguiente manera. La Sección 2 presenta los trabajos relacionados, examinando investigaciones recientes sobre detección de anomalías, enfoques no supervisados y mecanismos de respuesta aplicados a la seguridad de redes. La Sección 3 describe la metodología, incluyendo el entorno experimental, la captura y procesamiento del tráfico, la construcción de características, el modelo Isolation Forest (IF) y su integración con el mecanismo de decisión y control inline. La sección 4 presenta y discute los resultados considerando tanto el desempeño predictivo como las métricas de anticipación y operación del sistema. Finalmente, la Sección 5 expone las conclusiones, las principales limitaciones identificadas y las líneas de trabajo futuro.

2. Related Works

Faizin et al. combinaron XGBoost con una selección de características basada en información mutua y umbrales para distinguir tráfico normal y ataques donde analizaron variables relacionadas con el volumen de datos, la duración de las conexiones y el comportamiento TCP. Al reducir las características de 42 a 19, la exactitud aumentó del 80,29% al 87,63% y el F1 del 84,73% al 89,56% mientras que el tiempo de entrenamiento disminuyó de 6,07 a 2,11 segundos {12}. Almohaimeed y Albalwy seleccionaron 16 de las 83 características mediante la combinación de cinco métodos y aumentaron el F1 de un perceptrón multicapa del 72,2% al 91,9%. El conjunto seleccionado incluyó el conteo de banderas SYN y el puerto de destino, relacionados con ataques SYN flod y escaneo de puertos. Sin embargo, la sensibilidad disminuyó del 99,7% al 87,1% mostrando que la mejora del equilibrio entre precision y sensibilidad estuvo acompañada de una menor proporción de ataques detectados{13}. Ambos estudios muestran que la selección de características influye en la clasificación, pero no evalúan el ciclo completo de detección, decision y respuesta sobre tráfico en tiempo real.

Existen diversos modelos de aprendizaje no supervisado para detectar anomalías en el tráfico de red. Entre ellos, OCSVM permite aprender una frontera a partir del tráfico normal e identificar observaciones que se apartan de ese comporamiento. Bountzis et al. combinaron este modelo con un autoencoder y entrenaron el sistema únicamente con tráfico benigno donde también observaron que el ajuste de sus parámetros influye en la detección {3}. Wang et al. emplearon un autoencoder para extraer características y combinaron OCSVM con un modelo de mezcla gaussiana alcanzando un F1 de 98,85% y una tasa de falsos positivos de 2,75% frente al 90,84% y 25,02% cuando solo se usa OCSVM por separado bajo las mismas características extraídas {14}. En otro estudio, Wang et al. combinaron Random Forest con un autoencoder entrenado únicamente con tráfico normal, reduciendo la tasa media de falsos positivos del 8,31% al 1,81% en una de sus evaluaciones con ataques excluidos del entrenamiento. Random Forest identificaba las posibes amenazas mediante sus probabilidades de clasificación y el autoencoder revisaba las muestras señaladas para reclasificar como normales aquellas compatibles con el comportamiento benigno {15}.

Iturbe-Araya y Rifà-Pous compararon cuatro detectores de anomalías en escenarios de redes domésticas con dispositivos IoT en la que OCSVM obtuvo un F1 promedio de 92,97% frente al 93,86% de Local Outlier Factor (LOF) y al 83,07% de Isolation Forest (IF), considerando cuatro conjuntos de datos. Estos resultados situaron a OCSVM por encima de IF y ligeramente por debajo de LOF en la evaluación conjunta, sin demostrar una superioridad en todos los escenarios {16}. Pinto et al. compararon IF y OCSVM entrenados únicamente con datos normales en un sistema industrial de tratamiento de agua. IF destacó 19 de 26 escenarios de ataque y OCSVM 16, aunque varias detecciones fueron parciales {17}. Estas comparaciones muestran que el rendimiento de OCSVM frente a otros detectores varia según los datos y el escenario evaluado.

La evaluación de la detección debe considerer tanto los ataques identificados como el tráfico normal reconocido correctamente. Patel et al. compararon OCSVM e IF en redes domésticas y reportaron una identificación media de ataques del 91,75% para OCSVM, frente al 80,40% de IF. Sin embargo, IF reconoció correctamente una mayor proporción de tráfico normal: 86,75% frente al 84,87% de OCSVM, según los promedios de sus evaluaciones por característica {18}. También importa cómo se agrupan los eventos para analizarlos. Landauer et al. revisaron métodos que cuentan eventos en ventanas temporales y señalaron que estas deben ser suficientemente cortas para captar cambios y permitir una detección oportuna, pero suficientemente

amplias para evitar series con numerosos intervalos sin eventos {19}.

La utilidad operativa de la detección también depende de que sus resultados puedan traducirse en acciones sobre el tráfico. Florea y Craus integraron Snort con un controlador de red definida por software (SDN) para activar medidas de mitigación ante ataques DDoS. En una prueba de inundación TCP SYN el caudal del ataque descendió de 156,33 Mbps a 18,11 Mbps a los 10 segundos y se estabilizó en 3 Mbps a los 50 segundos {20}. Rabed et al. propusieron otro mecanismo que valida las solicitudes de conexión TCP e instala reglas de bloqueo en el switch conectado a la Fuente identificada como maliciosa. En sus pruebas de ataques SYN flood redujeron hasta un 39% el tiempo medio de respuesta de las solicitudes HTTP legítimas {21}. Ambos estudios conectan la detección con medidas de mitigación y evalúan sus efectos sobre el tráfico de ataque o el servicio ofrecido a usuarios legítimos.

La respuesta automática requiere establecer cuándo intervenir sobre el tráfico y si corresponde limitarlo o bloquearlo. Zhang Xu propuso un mecanismo de defensa ante ataques DDoS en la nube que activa medidas únicamente cuando la probabilidad de ataque supera 0,5 en dos ventanas consecutivas. La intervención comienza con la limitación del tráfico y puede avanzar al bloqueo temporal de direcciones IP según la gravedad del evento, buscando evitar interrupciones por una clasificación aislada {22}. Ma et al. Incorporaron un umbral adaptativo para detectar ataques de denegación de servicio de baja tasa, reduciendo los falsos positivos del 4,8% al 0,3% y aumentando el F1 de 0,976 a 0,985 respecto al umbral fijo. Su mecanismo también acumula evidencias de distintas observaciones para confirmar las fuentes sospechosas antes de instalar reglas de bloqueo {23}. Ambos estudios incorporan criterios de confirmación antes de intervenir, con el propósito de reducir bloqueos innecesarios del tráfico legítimo.

La evaluación de sistemas de detección y respuesta debe considerar cuánto tiempo transcurre desde el inicio del ataque hasta su identificación y la aplicación de una medida de mitigación. Puccetti y Ceccarelli compararon detectores ante ataques DoS sobre MQTT y reportaron una latencia media de detección de 6,49 s para XGBoost, frente a 7,78s de ExtraTreeClassifier {24}. Por su parte, Bhattacharya et al. combinaron estadísticas de flujo y registros del controlador con un análisis de entropía y divergencia de Kullback-Leibler en una arquitectura SDN. Ante ataques TCP SYN flood, su propuesta redujo el tiempo medio desde la llegada del primer paquete malicioso hasta la instalación de la regla de mitigación de 350 a 210 ms, y la tasa de falsos positivos del 5,8% al 1,5% frente a un detector basado únicamente en un umbral de entropía {25}. Estos trabajos destacan la importancia de precisar qué etapas comprende el tiempo reportado y evaluar la rapidez de la intervención junto con el riesgo de afectar el tráfico legítimo.

3. Methodology

La metodología comprende el diseño, la implementación y la evaluación de CyberFlow, un sistema de detección de anomalías de red y respuesta graduada. El sistema observa el tráfico mediante un espejo SPAN y combina Isolation Forest con reglas heurísticas para generar señales de detección. Estas señales se traducen en decisiones de permitir, limitar o bloquear.

El procedimiento se organiza en seis componentes: diseño del estudio y alcance experimental; arquitectura del sistema y entorno de red; desarrollo del detector híbrido; política de decisión y respuesta graduada; despliegue, monitoreo y actualización del detector; y protocolo de evaluación y validación del sistema. El desarrollo del detector comprende siete fases, desde la recolección de datos hasta la evaluación y el congelado de sus artefactos. Asimismo, se describe el procedimiento de replicabilidad y reentrenamiento con nuevos datos de referencia, cuya promoción requiere evaluar el modelo candidato y contar con aprobación humana. La Figura 1 presenta la organización general de la metodología y la relación entre sus componentes.

{Figura 1. Metodología general del sistema propuesto}

3.1 Diseño del estudio y alcance experimental

El estudio adopta un enfoque cuantitativo y aplicado, orientado al desarrollo y la evaluación de un sistema de detección de anomalías con respuesta graduada. Las pruebas se realizan en una red experimental segmentada, con tráfico legítimo generado mediante seis perfiles de uso y ataques controlados ejecutados en periodos identificados. Esta organización permite relacionar el tráfico observado con las señales del detector y las acciones de respuesta.

La evaluación considera el desempeño del detector y el funcionamiento del sistema integrado durante su operación. Se examinan la identificación de tráfico normal y anómalo, los falsos positivos, el

aporte del modelo y los heurísticos. Asimismo, se verifican la aplicación y reversión de las medidas de respuesta, los tiempos de detección y actuación y la afectación del tráfico legítimo. El alcance se limita a las condiciones de red, los perfiles y los escenarios ensayados, por lo que los resultados no establecen por sí solos su desempeño en otras redes.

3.2 Arquitectura del sistema y entorno de red

El entorno experimental integra máquinas virtuales conectadas a un conmutador físico, denominado CORE-STACK, y segmentos separados mediante VLAN. Los clientes y la estación atacante Kali Linux se ubican en la VLAN 20; el servidor protegido, en la VLAN 30 o zona desmilitarizada (DMZ); y la interfaz de gestión del sensor, en la VLAN 60. El enrutamiento entre segmentos corresponde a dos cortafuegos pfSense configurados con una dirección virtual compartida mediante CARP. La Figura 2 muestra la topología y distingue el recorrido del tráfico, su captura y la distribución de las decisiones de respuesta.

{Figura 2. Arquitectura del sistema y topología del entorno experimental}

La observación se realiza mediante una sesión SPAN que replica ambos sentidos de los enlaces troncales de pfSense hacia el sensor. Este dispone de dos interfaces: ens37, sin dirección IP y en modo promiscuo, recibe el tráfico espejado; y ens34, con dirección 10.10.60.11, permite la administración y comunicación de los servicios. Sobre la misma interfaz de captura operan tcpdump, que almacena paquetes en formato PCAP, y Suricata, que genera eventos de aplicación en eve.json. Ambas fuentes alimentan la extracción de características del detector.

El sensor ejecuta el motor de detección y publica las decisiones mediante un feed firmado digitalmente. El bastión VM-GESTION actúa como intermediario para distribuirlo hacia el host protegido, donde un agente verifica la firma y aplica las medidas mediante nftables. Esta separación permite observar el tráfico de forma pasiva y ejecutar la respuesta en el equipo de destino. El panel web proporciona acceso de solo lectura al estado y los registros del sistema. La Tabla 1 resume los componentes del entorno.

Table 1. Componentes, segmentos y funciones del entorno

Componente	Dirección o Identificación	Segmento	Función
Conmutador físico	CORE-STACK	Troncales VLAN	Conectar los segmentos y replicar el tráfico mediante SPAN
Firewall	pfSense-A y pfSense-B	Enrutamiento entre VLAN	Aplicar la segmentación y proporcionar las puertas de enlace
Clientes legítimos	10.10.20.21 - 10.10.20.26	VLAN 20	Generar tráfico correspondiente a seis perfiles de actividad
Estación atacante	Kali Linux – 10.10.20.30	VLAN 20	Ejecutar las pruebas de ataque controladas
Servidor protegido	srv-dmz – 10.10.30.10	VLAN 30	Alojar los servicios objetivo y palicar las acciones del agente
Sensor	Cyberflow-sensor 10.10.60.11	VLAN 60; captura por interfaz sin IP	Capturar tráfico, extraer características, ejecutar el detector y publicar el feed
Bastión	VM-GESTION – 10.10.10.30	VLAN 10	Proporcionar acceso administrativo y retransmitir el feed
Servicio DNS	10.10.10.20	VLAN 10	Resolver consultas de los clientes y recibir los ensayos DNS
Comparador por firmas	VM-SURICATA – 10.10.60.13	VLAN 60	Ejecutar Suricata 7.0.3 con ET Open para la comparación experimental

Suricata desempeña dos funciones diferenciadas: en el sensor genera la telemetría de aplicación consumida por CyberFlow, sin reglas de alerta activas; en la máquina comparadora utiliza reglas ET Open para evaluar la detección por firmas. El procedimiento de comparación se describe en la sección 3.6.

3.3 Desarrollo del detector híbrido

El detector combina un modelo Isolation Forest, ajustado a partir del tráfico de referencia de la red, con reglas heurísticas que identifican patrones específicos de actividad. Su desarrollo comprende siete fases: recolección de datos; preprocesamiento y extracción de características; partición del dataset; escalado y entrenamiento; calibración del umbral; integración de heurísticos; y evaluación y congelado. La Figura 3 presenta este procedimiento y distingue los datos utilizados para entrenar, calibrar y evaluar el detector.

{Figura 3. Fases de desarrollo del detector híbrido}

3.3.1 Recolección de datos

Los datos se obtienen mediante la captura del tráfico generado en el entorno descrito en la sección 3.2. El tráfico legítimo procede de seis perfiles de actividad: ofimática, con solicitudes web y consultas DNS; navegación, con acceso a recursos HTTP y HTTPS; descargas, con transferencias de mayor tamaño; aplicación, con solicitudes a servicios API; ligero, con comunicaciones de baja frecuencia; y errático, con ráfagas de actividad y periodos de silencio. Cada perfil utiliza una dirección de origen distinta dentro del rango 10.10.20.21–10.10.20.26, lo que permite observar su comportamiento por separado. La generación se realiza mediante programas de cliente y servidor, con selección ponderada de actividades, intervalos de espera aleatorios y variaciones de intensidad según el horario.

Las pruebas de ataque se ejecutan desde Kali Linux y comprenden escaneo de puertos, fuerza bruta HTTP, inundación HTTP y consultas DNS inusuales. Los ensayos registran la dirección de origen, el objetivo y el periodo de ejecución, lo que permite relacionar la actividad generada con las observaciones del sensor. Estas campañas proporcionan datos de ataque para evaluar la detección; sus escenarios y repeticiones se detallan en la sección 3.6.

El espejo SPAN entrega al sensor una copia del tráfico, de la que se obtienen dos fuentes complementarias: archivos PCAP mediante tcpdump y eventos HTTP, DNS y TLS mediante Suricata en eve.json. Ambas fuentes conservan marcas temporales y constituyen las entradas del preprocesamiento y la extracción de características.

3.3.2 Preprocesamiento y extracción de características

Las capturas PCAP y los eventos de aplicación se procesan para construir observaciones numéricas del comportamiento de cada entidad. Se excluyen las direcciones de gestión configuradas fuera del alcance y los protocolos de control CARP y pfsync. Asimismo, se eliminan las copias de paquetes introducidas por el espejo SPAN, identificadas mediante sus campos de protocolo, direcciones, puertos, identificador IP y secuencia, junto con su proximidad temporal. El procedimiento utiliza un intervalo de 5 ms y una diferencia de TTL no mayor que uno, conservando la copia de mayor TTL. Esta operación evita que las copias del espejo incrementen artificialmente los conteos o se interpreten como retransmisiones.

Los registros se agrupan por IP iniciadora y tiempo, generando una observación cada 10 s. Las características se calculan sobre ventanas retrospectivas de 10, 30 y 60 s, utilizando únicamente la información disponible hasta su cierre. Para una marca temporal (T), la ventana de duración (W) se define como:

$$v_W(T) = \{ r : T - W < t_r \le T \}, \quad W \in \{10,\ 30,\ 60\}\ s \tag{1}$$

Donde $v_W(T)$ es el conjunto de paquetes o eventos de una entidad incluidos en la ventana; (T) es el instante de cierre; (W) es su duración, que toma valores de 10, 30 o 60 s; (r) representa un registro y t_r su marca temporal. La condición $T - W < t_r \le T$ excluye el l[imite inicial e incluye el cierre, sin utilizar registros posteriores a (T). Para incorporar una fila al conjunto destinado al entrenamiento se exige una cobertura previa de 60 s. Durante la acumulación, la combinación de IP y cierre de ventana identifica cada fila y evita almacenarla nuevamente en ejecuciones sucesivas. Por ejemplo, si la ventana termina a las 10:01:00 y dura 60 segundos, considera los registros posteriores a 10:00:00 y hasta 10:01:00.

El extractor obtiene 31 características de las cuales el modelo utiliza las 28 correspondientes a las capas de red, transporte y aplicación; las 3 adicionales de capa de enlace quedan fuera por el alcance actual del presente trabajo. Las variables describen volumen, diversidad de destinos, comportamiento de las conexiones y actividad HTTP, DNS y TLS. La Tabla 2 presenta sus definiciones resumidas, ventanas y unidades.

La variable 26 permanece en cero porque la configuración de Suricata utilizada no permite observar los fallos de negociación TLS. Se conserva para mantener la estructura de 28 entradas del modelo; su valor no implica ausencia de fallos.

Cuando el denominador de una razón es cero, se asigna el valor 0 y se conservan contadores

auxiliares para distinguir la ausencia de actividad de una proporción efectivamente nula. Los identificadores, las marcas temporales y estos contadores se almacenan como metadatos, sin incorporarse a las entradas del IF. Las características HTTP y DNS dependen del tráfico visible en claro, mientras que las de TLS utilizan metadatos observables sin descifrar su contenido. El escalado de las variables se realiza posteriormente, durante la fase de entrenamiento.

Tabla 2. Características utilizadas como entrada del modelo

N.°	Característica	Capa	Ventana(s)	Unidad
1	Tasa de paquetes IPv4	L3	10	Paquetes/s
2	Tasa de bytes IPv4	L3	10	Bytes/s
3	Longitud media de los paquetes IPv4	L3	10	Bytes
4	Proporción de paquetes IPv4 de 500-1500 bytes	L3	10	Proporción
5	Razón de IP de destino distintas respecto a intentos de flujo	L3	30	Proporción
6	Proporción de paquetes ICMP	L3	10	Proporción
7	TTL medio de los paquetes IPv4	L3	10	Valor de TTL
8	Proporción de paquete IPv4 fragmentados	L3	10	Proporción
9	Razón de protocolos IP distintos respecto al número de paquetes	L3	30	Proporción
10	Tasa de nuevos intentos de flujo	L4	10	Intentos/s
11	Tasa de paquetes SYN salientes sin ACK	L4	10	SYN/s
12	Razón de SYN-ACK recibidos respecto a SYN enviados, acotada a uno	L4	10	Proporción
13	Proporción de paquetes TCP con bandera RST	L4	10	Proporción
14	Razón de puertos de destino distintos respecto a intentos con puerto	L4	30	Proporción
15	Proporción de segmentos TCP con carga y secuencia repetida	L4	10	Proporción
16	Duración media observada de los flujos hasta el cierre de ventana	L4	30	Segundos
17	Proporción de bytes transmitidos respecto al total transmitido y recibido	L4	30	Proporción
18	Proporción de transacciones HTTP con estado ≥ 400	L7	60	Proporción
19	Razón de respuestas NXDOMAIN respecto a consultas DNS, acotada a uno	L7	60	Proporción
20	Tasa de sesiones TLS observadas	L7	60	Sesiones/s
21	Tasa de solicitudes HTTP	L7	60	Solicitudes/s
22	Entropía de los métodos HTTP	L7	60	Bits
23	Proporción de transacciones HTTP con estado 401 o 403	L7	60	Proporción
24	Tasa de consultas DNS	L7	60	Consultas/s
25	Razón de nombres DNS distintos respecto a consultas	L7	60	Proporción
26	Indicador de fallos de negociación TLS basado en ausencia de versión*	L7	60	Proporción
27	Proporción de eventos TLS 1.3 entre aquellos con versión conocida	L7	60	Proporción
28	Proporción de transacciones HTTP con estado 500-599	L7	60	Proporción

3.3.3 Partición del dataset

La base considerada para la partición comprende 337 799 observaciones de tráfico

normal. Estas se organizan en bloques temporales de dos horas y se distribuyen de forma determinista entre entrenamiento, validación y prueba. La asignación sigue un patrón cíclico de cinco bloques: entrenamiento, entrenamiento, validación, entrenamiento y prueba. Este procedimiento establece una distribución aproximada de 60%, 20% y 20%, permitiendo incorporar distintas franjas horarias en los tres subconjuntos.

Debido a que las características utilizan hasta 60 s de información retrospectiva, las observaciones cercanas al límite entre bloques pueden compartir paquetes o eventos. Para evitar este solapamiento entre subconjuntos, se aplica una banda de guarda de 60 s al inicio de cada bloque posterior al primero. Este criterio deja fuera a 2 778 observaciones, por lo que el conjunto finalmente utilizado comprende 335 021 observaciones.

El subconjunto de entrenamiento se emplea para ajustar el escalador y el Isolation Forest; el de validación para calibrar el umbral de anomalía; y el de prueba para estimar la tasa de falsos positivos sobre tráfico normal. La validación contiene 65 633 ventanas normales, incluidas 80 ventanas de la entidad Kali, de las cuales 17 presentan actividad observable. No se encontró evidencia independiente que permita confirmar que esas 17 ventanas sean ataques; por tanto, su normalidad no está verificada. La Tabla 3 presenta la composición de la base y su distribución final.

Tabla 3. Composición y partición de los datos normales

Componente	Número de observaciones	Propósito
Base considerada para la partición	337 799	Observaciones antes del descarte por banda de guarda
Descartadas por banda de guarda	2 778	Evitar el solapamiento temporal entre subconjuntos
Entrenamiento	203 967	Ajustar el escalador y el Isolation Forest
Validación	65 633	Calibrar el umbral; incluye 80 ventanas Kali, 17 con actividad observable y normalidad no verificada
Prueba	65 421	Estimar la tasa de falsos positivos
Total utilizado	335 021	Suma de entrenamiento, validación y prueba

Los datos de ataque se mantienen separados del ajuste del escalador, del entrenamiento del Isolation Forest y de la calibración del umbral. Se utilizan para evaluar inicialmente la capacidad de detección del IF y, posteriormente, comparar su desempeño con el de los heurísticos y el detector combinado. Los escenarios y las condiciones experimentales se detallan en la sección 3.6.

3.3.4 Escalado y entrenamiento

Las 28 características se organizan en matrices numéricas, conservando el mismo orden en entrenamiento, validación y prueba. Se aplica StandardScaler utilizando la media y la desviación estándar calculadas exclusivamente con las 203 967 observaciones normales de entrenamiento. La transformación se define como, Eq. (2):

$$z_{ij} = \frac{x_{ij} - \mu_j}{s_j}, \qquad s_j = \begin{cases} \sigma_j, & \sigma_j > 0, \\ 1, & \sigma_j = 0 \end{cases} \tag{2}$$

Donde x_{ij} es el valor de la característica j en la observación i; μ_j y σ_j son su media y desviación estándar en entrenamiento; s_j es el factor de la escala la cual utiliza la desviación estándar, si esta es cero utiliza 1 para evitar dividir entre cero y z_{ij} es el valor de la característica después de aplicar la estandarización.

El IF se ajusta únicamente con las observaciones normales estandarizadas para representar el comportamiento de referencia. Construye árboles mediante particiones aleatorias; las observaciones aisladas con recorridos más cortos se consideran potencialmente anómalas. Se utilizan 500 árboles, todas las características disponibles para cada árbol y la semilla aleatoria 20260813 para favorecer la reproducibilidad.

El parámetro de contaminación se establece en modo automático y el umbral operativo se calibra posteriormente con validación, como se describe en la sección 3.3.5. Se conservan el escalador, el bosque y el orden de las características para procesar nuevas observaciones. La función score_samples asigna puntuaciones menores a las observaciones con mayor anomalía.

3.3.5 Calibración del umbral

El modelo ajustado puntúa las 65 633 observaciones normales de validación, transformadas con el escalador obtenido en entrenamiento. Se adopta α = 0,05 como criterio de calibración y se establece el umbral en el percentil 5 de las puntuaciones, Eq. (3). La validación incluye 80 ventanas de Kali, 17 con actividad observable, sin confirmación independiente de que correspondan a ataques.

$$\tau = Q_\alpha\left(\left\{s(z_i)\right\}_{i=1}^{n_v}\right), \qquad \alpha = 0{,}05$$

Donde θ es el umbral; qα es el cuantil de orden α; s_i es la puntuación de normalidad asignada por el IF a la observación estandarizada i; i es su índice y n el número de observaciones de validación. Este criterio sitúa aproximadamente el 5% de las observaciones normales de validación por debajo del corte, sin utilizar datos de prueba ni etiquetas de ataque para seleccionarlo. La composición de validación contiene actividad de Kali cuya normalidad no está verificada, por lo que α = 0,05 reproduce el criterio operativo, pero no prueba una tasa futura ni un óptimo.

El percentil 5 produce un umbral de −0,069397 en la función de decisión del IF. Con la configuración automática del parámetro de contaminación, este valor equivale a −0,569397 en la escala de puntuación de normalidad, mediante la resta de 0,5. Ambas escalas generan la misma clasificación; la conversión corresponde al desplazamiento de salida del modelo y no a la estandarización de las características. Eq (4),

$$\tau = \tau_d - 0{,}5 = -0{,}069397 - 0{,}5 = -0{,}569397 \qquad (4)$$

Donde τ_d es el umbral en la escala de la función de decisión y t su equivalente en la escala de puntuación de normalidad del IF. Esta conversión conserva la clasificación y corresponde al desplazamiento de salida del modelo, no a la estandarización de las características La señal de anomalía se define mediante la ecuación (5),

$$A_{IF}(z) = \begin{cases} 1, & s(z) < \tau, \\ 0, & s(z) \geq \tau \end{cases}$$

Donde z es una observación estandarizada y $s(z)$ su puntuación. La señal $A_{IF}(z)$ toma el valor 1 cuando se identifica una anomalía y 0 en caso contrario.

El umbral permanece fijo durante la evaluación posterior. El valor α expresa el criterio adoptado para validación; no garantiza un 5% de falsos positivos en datos nuevos. Esta tasa se estima con la prueba normal, mientras que los datos de ataque permiten evaluar la capacidad de detección. La evaluación de prueba es reutilizada y no constituye un test ciego independiente.

3.3.6 Integración de heurísticos

El detector incorpora reglas deterministas que complementen las señales del IF. Estas se aplican sobre las características y los contadores originales de cada ventana, sin estandarización. Sus umbrales se establecen mediante criterios explícitos, configurables y versionados; no se entrenan ni se calibran estadísticamente como el umbral del IF.

La regla de fuerza bruta exige al menos cinco solicitudes HTTP en 60 s y una proporción de respuestas 401 o 403 igual o superior a 0,80. El escaneo se identifica mediante al menos 20 intentos de flujo en 30 s, una razón de puertos distintos de al menos 0,45 y una razón de respuestas SYN-ACK respecto a SYN no mayor que 0,30; alternativamente, se activa con al menos 200 intentos y una razón SYN-ACK/SYN no mayor que 0,10. El volumen HTTP elevado requiere al menos 100 solicitudes en 60 s y una proporción de fallos de autenticación inferior a 0,80. La regla DNS exige al menos 20 consultas en 60 s y una razón de respuestas NXDOMAIN de al menos 0,50 o de nombres distintos de al menos 0,45.

La detección combinada se expresa mediante una disyunción lógica: basta con que el IF o alguna regla identifique actividad sospechosa. Eq (6),

$$A_H(x) = A_{IF}(z) \quad \vee ¿ k = 1 ¿ 4\, h_k(x)$$

Donde $A_H(x)$ es la señal del detector híbrido; x, la observación con sus valores originales; z, su versión estandarizada; $A_{IF}(z)$, la señal del IF; y $h_k(x)$, la señal de la regla k. Cada señal vale 1 si se activa y 0 en caso contrario. El símbolo $\vee$ representa la disyunción lógica: el detector híbrido toma el valor 1 cuando el IF, alguno de los heurísticos o ambos identifican una anomalía. Toma el valor 0 cuando ninguno se activa.

La integración conserva la puntuación y el umbral del IF y registra el origen de las señales para evaluar cada componente por separado.

3.3.7 Evaluación y congelado

La evaluación mantiene fijos el escalador, el IF y el umbral calibrado. Se utilizan las 65 421 observaciones de prueba normal y 84 ventanas etiquetadas como actividad de ataque para estimar el desempeño. La evaluación es reutilizada: el conjunto ya participó en decisiones de desarrollo y no constituye un test ciego independiente. La unidad primaria es la ventana temporal de 10 s; la cobertura por episodio se presenta por separado. Se comparan el IF, los heurísticos y su combinación sobre los mismos datos de cada ensayo, mediante Eq. (7).

$$TPR = \frac{TP}{TP + FN}, \qquad FPR = \frac{FP}{FP + TN}$$

Donde TP son los ataques detectados; FN los ataques no detectados; FP las observaciones normales señaladas como anómalas y TN las normales correctamente identificadas. Se distingue la detección por ventana de la cobertura por episodio y se declara el denominador de cada resultado. Para el IF también se calcula ROC-AUC utilizando el negativo de su puntuación de normalidad, de modo que los valores mayores representen mayor anomalía.

El congelado conserva el escalador, el bosque, el umbral con su precisión completa, el esquema y orden de las variables, y la versión de las reglas. Un manifiesto registra los parámetros, las versiones de las bibliotecas y las huellas SHA-256 de los artefactos. Antes del despliegue se verifica que el modelo guardado mantenga las mismas decisiones y que el orden de sus entradas coincida con el extractor.

3.4 Política de decisión y respuesta graduada

La política transforma las señales del detector en acciones temporales por dirección IP. Una anomalía identificada únicamente por el IF genera una limitación de tráfico; las reglas de fuerza bruta y escaneo generan bloqueo, mientras que las de volumen HTTP elevado y actividad DNS inusual generan limitación. Cuando coinciden varias señales, prevalece BLOCK sobre LIMIT. La Tabla 4 resume esta correspondencia

Tabla 4. Correspondencia entre señales y acciones

Señal	Condición	Acción
Isolation Forest	Puntuación interior a -0,569397	ALERT→LIMIT
Heurístico de fuerza bruta HTTP	Se cumplen las condiciones de la regla	BLOCK
Heurístico de escaneo de puertos	Se cumple alguna de sus dos condiciones	BLOCK
Heurístico de volumen HTTP elevado	Se cumplen las condiciones de la regla	LIMIT
Heurístico de actividad DNS inusual	Se cumplen las condiciones de la regla	LIMIT
Sin detección	El IF no señala anomalía y ningún heurístico se activa	PERMIT

ALERT representa una señal de detección, mientras que LIMIT y BLOCK son acciones de respuesta. LIMIT descarta el tráfico que supera la tasa configurada; BLOCK descarta todo el tráfico procedente de la IP hacia el host protegido. PERMIT indica que la observación no genera una nueva restricción, pero no revoca una medida todavía vigente.

LIMIT tiene una duración de 300 s. Para BLOCK se aplican 300 s en el primer episodio, 1 800 s en el segundo y 3 600 s desde el tercero, con una marca para revisión humana. Las detecciones de bloqueo separadas por no más de 120 s se agrupan en un mismo episodio para evitar contar su republicación como reincidencia. El contador se reinicia tras más de 24 h sin actividad sancionada y no se generan bloqueos permanentes automáticos.

Las acciones se distribuyen mediante un feed firmado y se aplican con nftables en los hosts protegidos. Cada restricción conserva su caducidad, aunque deje de aparecer en el feed; un LIMIT posterior no reduce un BLOCK activo. Las medidas terminan por vencimiento o retirada manual. Los gateways, el servidor DNS, el sensor y el bastión permanecen en una lista de exclusión. Si el feed no está disponible o su firma es inválida, no se aplican nuevas acciones y las existentes conservan su caducidad.

3.5 Despliegue, monitoreo y actualización del detector

El detector se ejecuta en el sensor mediante servicios gestionados por systemd y procesa las observaciones cada 10 s. Las decisiones se distribuyen mediante un feed firmado, transferido

por el bastión hacia los agentes de los hosts protegidos. La interfaz de captura recibe el tráfico SPAN sin intervenir en su recorrido original; las restricciones se aplican en los hosts mediante nftables.

El monitoreo utiliza registros de decisiones y un panel de solo lectura, protegido mediante HTTPS, autenticación y roles. Se conservan la entidad observada, el cierre de ventana, la puntuación, el umbral y el origen de la detección para relacionar las señales con las respuestas. La replicación requiere adaptar las interfaces, redes, exclusiones y rutas, conservar las versiones de las dependencias y verificar los artefactos y el orden de las características.

El procedimiento de actualización contempla lotes diarios de nuevos datos, depurados antes de incorporarlos al entrenamiento. Se excluye la actividad marcada como sospechosa y se comprueba la distribución de cada lote frente a un conjunto fijo de referencia normal, separado del entrenamiento. Los datos nuevos pueden representar como máximo el 30 % del entrenamiento y ninguna IP puede aportar más del 10 % de esos datos nuevos.

El reentrenamiento se configura con una cadencia de 30 días, anticipable ante deriva sostenida durante tres días o por solicitud manual, con un mínimo de siete lotes aceptados. Cada candidato se compara con el modelo vigente utilizando los mismos datos normales de referencia y de ataque. Se exige que su tasa de falsos positivos no supere la del vigente en más de 0,5 puntos porcentuales y que su detección no disminuya. Sin datos de ataque, la evaluación se considera incompleta. La activación requiere aprobación humana y conserva una copia de la configuración anterior para permitir la reversión.

3.6 Protocolo de evaluación y validación del sistema

La evaluación distingue el desempeño del detector y la aplicación de la respuesta. Cada ensayo registra el escenario, origen, destino, periodo de ejecución, capturas y decisiones generadas. Las comparaciones utilizan los mismos datos por ensayo y conservan la identificación del modelo, umbral y versión de los heurísticos.

La validación de respuesta contrasta las decisiones del sensor con las entradas del feed y las restricciones registradas en nftables. Se distingue la aplicación automática de extremo a extremo de la prueba aislada del mecanismo de bloqueo, evitando atribuir a ambas el mismo alcance de validación.

Tabla 5. Diseño de los experimentos y criterios de evaluación

Evaluación	Datos o escenarios	Alcance	Criterios
Falsos positivos del IF	Tráfico normal de prueba	65 421 ventanas	FPR
Detección del IF	Ventanas de actividad etiquetadas por familia proxy observable	65 505 ventanas; evaluación reutilizada, no test ciego independiente	Precisión, recall, F1, FPR, ROC-AUC y AP
Comparación por componentes	DNS inusual, inundación HTTP y escaneo de puertos	Tres episodios por familia; nueve en total	Cobertura del IF, heurísticos y combinado
Comparación con Suricata	Reproducción de los mismos PCAP con reglas ET Open	Nueve episodios y un control adicional con Nikto	Cobertura y latencia de detección
Fuerza bruta HTTP	Solicitudes con fallos de autenticación	Un ensayo adicional	Activación del heurística y acción generado
Aplicación de respuestas	Ensayo automático de LIMIT y prueba aislada de BLOCK	Registros del motor, feed y host protegido	Correspondencia entre decisión y restricción aplicada

La comparación por componentes utiliza los mismos episodios para identificar el aporte del IF y de los heurísticos. Un episodio se considera detectado cuando se registra al menos una señal durante su ejecución, asociada a actividad etiquetada por familia proxy observable. Esta cobertura se presenta separada de la detección por ventana. La comparación con Suricata requiere los mismos PCAP y declarar las reglas y el alcance de inspección.

En los ensayos con registros temporales completos, las latencias se calculan como. Eq (8),

$$L_{det} = t_{det} - t_0, \qquad L_{resp} = t_{apl} - t_{det} \tag{8}$$

Donde t_0 es el inicio del ataque; t_{det} es la primera detección y t_{apl} la aplicación de la medida en el host. L_{det} representa el tiempo hasta detectar el ataque y L_{resp} el tiempo adicional hasta aplicar la respuesta. Los casos sin detección o sin registro de aplicación se identifican por separado, sin asignarles una latencia igual a cero.

4. Resultados y discusión

4.1 Evaluación del detector Isolation Forest

Se evaluó el modelo operativo if_limpio_2026_10 sobre ventanas temporales de 10 s por entidad. El conjunto contiene 65 421 ventanas normales de prueba y 84 ventanas etiquetadas como ataque, para 65 505 observaciones. El umbral congelado corresponde a score_samples < −0,5693969936322356. La evaluación es reutilizada y no constituye un test ciego independiente.

La calibración se realizó sobre 65 633 ventanas de validación. Este conjunto incluye 80 ventanas de Kali, 17 con actividad observable, sin evidencia independiente que permita confirmar que correspondan a ataques. Se conservan para describir la calibración operativa, pero su normalidad no está verificada.

Tabla 6. Desempeño del IF limpio sobre la evaluación de 65 505 ventanas.

Indicador	**Valor**
TP	54
FP	2 961
TN	62 460
FN	30
Precisión	1,7910 %
Recall	64,2857 %
F1	0,0348
FPR	4,5261 %
ROC-AUC	0,9247
AP	0,1858

El IF detectó 54 de 84 ventanas etiquetadas como ataque y produjo 2 961 falsas alarmas sobre 65 421 ventanas normales. La precisión fue 1,79 %. La prevalencia fue 0,128 %, por lo que un FPR de 4,53 % genera muchas más alarmas normales que verdaderos positivos. ROC-AUC resume la capacidad de ordenamiento y no garantiza una buena precisión en el umbral operativo.

Tabla 7. Comparación de configuraciones sobre las mismas 65 505 ventanas.

Configuración	Entrenamiento	Precisión	Recall	F1	FPR	ROC-AUC	AP
IF limpio (desplegado)	esta red	1,79 %	64,29 %	0,0348	4,53 %	0,9247	0,1858
IF recalibrado (contaminado, antecedente)	esta red	1,82 %	64,29 %	0,0354	4,45 %	0,9228	0,2026
IF scaled (lab)	laboratorio	0,39 %	91,67 %	0,0077	30,34 %	0,8250	0,0043
IF primary (lab)	laboratorio	0,36 %	82,14 %	0,0072	29,07 %	0,8283	0,0044
IF uniform (lab)	laboratorio	0,32 %	85,71 %	0,0064	34,13 %	0,7960	0,0054
IF exact (lab; idéntico a uniform)	laboratorio	0,32 %	85,71 %	0,0064	34,13 %	0,7960	0,0054
OCSVM (lab)	laboratorio	0,13 %	100,00 %	0,0026	99,57 %	0,4555	0,0011
LOF (lab)	laboratorio	0,12 %	70,24 %	0,0023	76,80 %	0,2991	0,0009
Elliptic Envelope (lab)	laboratorio	0,06 %	42,86 %	0,0013	86,90 %	0,3443	0,0014

Los candidatos de laboratorio y los modelos entrenados con datos de esta red tienen distinto origen; el resultado mide transferencia al entorno objetivo, no superioridad intrínseca. IF limpio e IF contaminado detectaron ambos 54/84, pero difirieron en falsas alarmas, ROC-AUC y AP. No se afirma equivalencia estadística. IF uniform e IF exact tienen idénticos scores: son nueve filas, pero ocho configuraciones distintas.

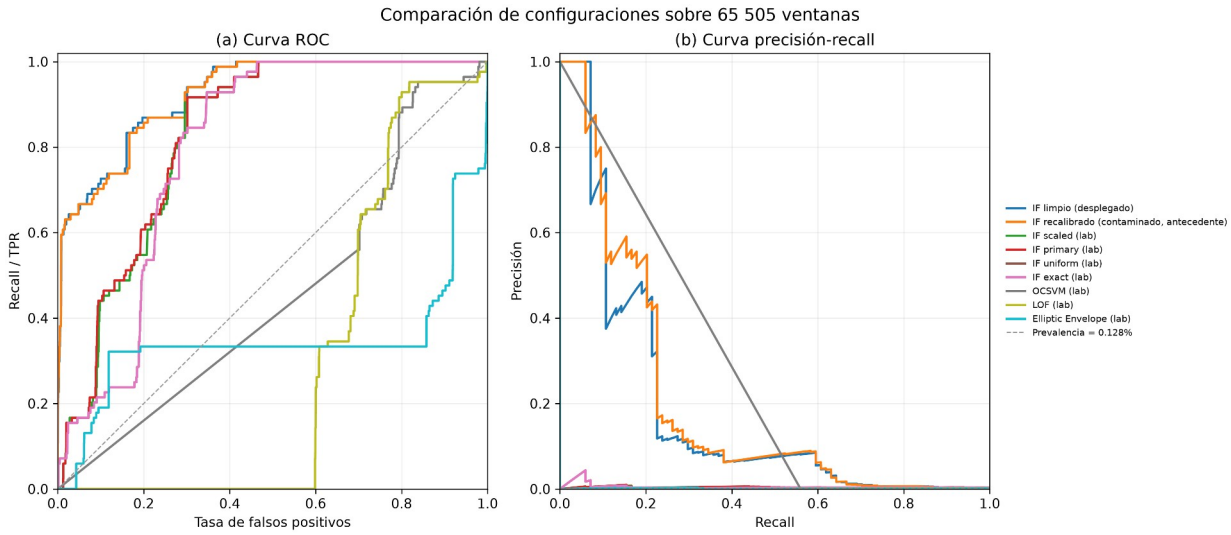


Figura 4. Curvas ROC y precision-recall de las configuraciones evaluadas.

La figura resume el comportamiento de ranking usando los scores de precisión completa y la prevalencia real de 0,128 % como referencia. No reemplaza la decisión en el umbral congelado: una curva favorable puede coexistir con una precisión operativa baja.

La sensibilidad de α se calculó con las 65 633 ventanas de validación originales; α = 0,05 reproduce el umbral operativo, sin demostrar que sea óptimo. En el test normal, las ventanas con paquetes recientes tuvieron 2 837 falsos positivos entre 19 863 ventanas (14,28 %); las ventanas sin paquetes recientes pero con actividad retrospectiva tuvieron 124 entre 12 208 (1,02 %); las 33 350 restantes no tuvieron falsos positivos. El FPR global depende de esta composición. Offline se puntúan todas las ventanas; en vivo, las ventanas sin paquetes omiten el IF pero aún pueden activar heurísticos. Por ello, estos resultados no equivalen al FPR operativo del sistema completo.

Como limitaciones, la evaluación es reutilizada; las ventanas de Kali no tienen confirmación independiente de ser ataques; las familias son proxies observables; la evaluación de esta sección

corresponde al IF solo y el híbrido se analiza posteriormente; y los resultados no se extrapolan automáticamente a otra red o a IPv6.

Referencias

{1} https://doi.org/10.1002/ett.4150

{2} https://doi.org/10.22266/ijies2026.0531.16

{3} https://doi.org/10.1371/journal.pone.0263423

{4} https://doi.org/10.1002/spy2.70084

{5} https://doi.org/10.1007/s44163-025-00578-1

{6} https://doi.org/10.1002/cpe.8034

{7} https://doi.org/10.1016/j.future.2025.108314

{8} https://doi.org/10.11591/eei.v13i4.7143

{9} https://doi.org/10.1109/ACCESS.2024.3393998

{10} https://doi.org/10.3390/electronics14071289

{11} https://doi.org/10.1007/s44443-026-01098-3

{12} https://doi.org/10.22266/ijies2024.0630.18

{13} https://doi.org/10.3390/app142411966

{14} https://doi.org/10.3390/electronics12040930

{15} https://doi.org/10.3390/sym15030568

{16} https://doi.org/10.1007/s10207-024-00961-6

{17} https://doi.org/10.1007/s44196-024-00644-z

{18} https://doi.org/10.3390/electronics9060923

{19} https://doi.org/10.1007/s10207-024-00921-0

{20} https://doi.org/10.1007/s10207-025-01012-4

{21} https://doi.org/10.1186/s42400-025-00514-x

{22} https://doi.org/10.1007/s43926-026-00372-0

{23} https://doi.org/10.1007/s44443-026-00635-4

{24} https://doi.org/10.1007/s10664-025-10766-3

{25} https://doi.org/10.1007/s44354-026-00020-7

Referencias

[1] Z. Ahmad, et al., «Network intrusion detection system: A systematic study of machine learning and deep learning approaches», Transactions on Emerging Telecommunications Technologies, Vol. 32, No. 1, 2021, doi: 10.1002/ett.4150.

[2] E. F. Agyemang, «Anomaly detection using unsupervised machine learning algorithms: A simulation study», Scientific African, 2024, doi: 10.1016/j.sciaf.2024.e02386.

[3] M. K. Islam, et al., «Network anomaly detection using LightGBM: A gradient boosting classifier», In: Proc. of the 30th International Telecommunication Networks and Applications Conference, 2020, doi: 10.1109/ITNAC50341.2020.9315049.

[4] S. Ness, et al., «Anomaly detection in network traffic using advanced machine learning techniques», IEEE Access, 2025, doi: 10.1109/ACCESS.2025.3526988.

[5] R. Rabih, et al., «Highly accurate anomaly based intrusion detection through integration of the local outlier factor and convolutional neural network», Scientific Reports, Vol. 15, 2025, doi: 10.1038/s41598-025-08175-z.

[6] K. Luong, et al., «ConceptUML: Multiphase unsupervised threat detection via latent concept learning, hidden Markov models and topic modelling», Journal of Information Security and Applications, 2025, doi: 10.1016/j.jisa.2025.104160.

[7] L. Santos, et al., «A flow-based intrusion detection framework for internet of things networks», Cluster

Computing, 2021, doi: 10.1007/s10586-021-03238-y.

[8] A. A. Abdullah y D. B. Abdullah, «A unified framework for real-time intrusion detection and mitigation using Apache Spark», Journal of Wireless Mobile Networks, Ubiquitous Computing and Dependable Applications, 2025, doi: 10.58346/JOWUA.2025.I4.037.

[9] S. Dash y J. M. Acken, «Intrusion detection latency: the neglected metric», Cybersecurity, Vol. 9, 2026, doi: 10.1186/s42400-026-00574-7.

[10] F. Zwayed, et al., «An efficient intrusion detection systems in fog computing using forward selection and BiLSTM», Bulletin of Electrical Engineering and Informatics, Vol. 13, No. 4, 2024, doi: 10.11591/eei.v13i4.7143.

[11] A. Aljammal, et al., «Anomaly intrusion detection using machine learning - IG-R based on NSL-KDDdataset», Bulletin of Electrical Engineering and Informatics, Vol. 13, No. 6, 2024, doi: 10.11591/eei.v13i6.7308.

[12] G. Rapate, et al., «Hybrid AI-driven anomaly detection and sequential attack classification for securing IoT networks», Bulletin of Electrical Engineering and Informatics, Vol. 15, No. 1, 2026, doi: 10.11591/eei.v15i1.11048.

[13] N. Alassaf, et al., «Detecting anomalies in MQTT/MQTT-SN traffic using intelligent learning models», Bulletin of Electrical Engineering and Informatics, Vol. 15, No. 3, 2026, doi: 10.11591/eei.v15i3.11889.

[14] W. H. M. Kurdi, et al., «Efficient two-stage intrusion detection system based on hybrid feature selection techniques and machine learning classifiers», International Journal of Intelligent Engineering and Systems, Vol. 18, No. 3, 2025, doi: 10.22266/ijies2025.0430.16.

[15] M. B. Mortatha y D. A. Abdulah, «Hybrid WOA-boosting and GNN framework for multi-protocol IoT intrusion detection and attack classification using the CICIoMT-2024 dataset», International Journal of Intelligent Engineering and Systems, Vol. 19, No. 6, 2026, doi: 10.22266/ijies2026.0630.25.

[16] T. Shivakumara, et al., «Efficient intrusion detection with DL-Protect+: Balancing accuracy, interpretability, and resource efficiency», International Journal of Intelligent Engineering and Systems, Vol. 19, No. 3, 2026, doi: 10.22266/ijies2026.0331.01.

[17] S. Abdulwahab, et al., «Real-time malware prevention using gradient boosted decision trees on the EMBER 2024 dataset: A static analysis approach», International Journal of Intelligent Engineering and Systems, Vol. 19, No. 6, 2026, doi: 10.22266/ijies2026.0630.40.